

DSA

LAB MANUAL

Submitted to	
Name	Dr. Rajkumar Batchu
Department	CSE
Designation	Assistant Professor
Submitted by	
Name	Y.VAMSI
Roll No	AV.SC.U4CSE25154
Year/Sem/Section	2nd year / Sem-3 / CSE-B

INDEX

LAB	QUESTIONS	PGNO	REMARKS
1	**a)** A rocket launch system displays a countdown before launch. Instead using loop implement the countdown from n to 1, followed by "Launch!". **b)** Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power) where, P = principle growth factor, n = number of years. **c)** An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list. **d)** In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number. **e)** Write a recursive python program to print the first n terms of fibonacci series.	**4-8**	
2	a) Program to implement LINEAR SEARCH b) Program to implement BINARY SEARCH (1) With Sorted List: (2) Without Sorted List:	**9-11**	
3	a) Write a Program for BUBBLE SORT b) Write a program for INSERTION SORT c) Write a program for SELECTION SORT;	**12-14**	
4	a) Write a Program for Quick SORT	**15-17**	

	b)Write a Program for MERG SORT:		
5	a)Write a Program to implement single Linked List: b)Write a Program to implement Double Linked List: c)write a program to implement circular Linked List:	**18-26**	
6	a)Write python program to implement stack using Array b)Write python program to implement stack using LinkedList	**26-31**	

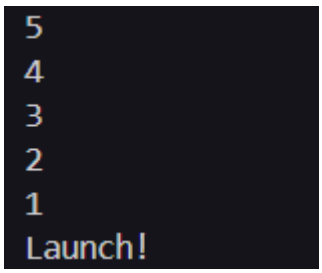
LAB-1

a) A rocket launch system displays a countdown before launch. Instead using loop implement the countdown from n to 1, followed by "Launch!".

CODE:

```
1. def countdown(n):
2.     if n <= 0:
3.         print("Launch!")
4.     else:
5.         print(n)
6.         countdown(n - 1)
7.
8. countdown(5)
```

OUTPUT:



EXPLANATION:

Base Case: When the number n reaches 0 or less, the recursion stops and the program prints "Launch!".

Recursive Step: If n is greater than 0, the program prints the current number and calls itself with n - 1, moving one step closer to the base case.

b) Banks calculate the growth of an investment using compound interest. The formula requires calculating powers, which can be implemented recursively. Write a recursive program to calculate P^n (power) where, P = principle growth factor, n = number of years.

CODE:

```
1. def calculate_power(P, n):
2.     if n == 0:
3.         return 1
4.     else:
5.         return P * calculate_power(P, n - 1)
6.


7. print(calculate_power(20, 5))
```

OUTPUT:

3200000

EXPLANATION:

Base Case: Any number raised to the power of 0 is 1. When n reaches 0, the function returns 1 and stops recurring.

Recursive Step: The function multiplies the base P by the result of the same function called with n - 1, effectively chaining the multiplication n times.

c) An HR department stores employee ID's in a list. Develop a recursive program to search for a given employee ID. Write a recursive function to search for an employee ID in a list.

CODE:

```
1. def search_employee(emp_list, emp_id, index=0):
2.     if index == len(emp_list):
3.         return False
4.     if emp_list[index] == emp_id:
5.         return True
6.     return search_employee(emp_list, emp_id, index + 1)
7.
8. employees = [101, 105, 108, 112]
9. print(search_employee(employees,  108))
```

OUTPUT:

True

EXPLANATION:

Base Cases: The function stops and returns False if the index goes out of bounds (meaning the entire list was searched). It returns True if the item at the current index matches the emp_id.

Recursive Step: If the current item is not a match, the function calls itself while incrementing the index by 1 to check the next element in the list.

ERROR TABLE:

SNO	ERROR	SOLUTION
1	Line 2: if index len(emp_list): is missing the equality operator	Change to if index == len(emp_list):

d) In a logistics company, the number of possible ways to arrange n different parcels are calculated using the factorial formula (n!). Develop a recursive program to calculate the factorial of a given number.

CODE:

```
1. def calculate_factorial(n):
2.     if n == 0 or n == 1:
3.         return 1
4.     else:
5.         return n * calculate_factorial(n - 1)
6.
7. print(calculate_factorial(5))
```

OUTPUT:

120

EXPLANATION:

Base Case: By mathematical definition, the factorial of 0 or 1 is 1. When n hits either of these, the recursion halts and returns 1.

Recursive Step: The function multiplies the current integer n by the factorial of n - 1, evaluating to $n \times (n-1) \times (n-2) \ldots \times 1$ as it resolves back up the call stack.

e) Write a recursive python program to print the first n terms of fibonacci series.

CODE:

```
1. def fibonacci(n):
2.     if n <= 0:
3.         return 0
4.     elif n == 1:
5.         return 1
6.     else:
7.         return fibonacci(n - 1) + fibonacci(n - 2)
8.
9. def print_fibonacci_series(terms):
10.     for i in range(terms):
11.         print(fibonacci(i), end=" ")
12.     print()
13.
14. print_fibonacci_series(7)
```

OUTPUT:

0 1 1 2 3 5 8

EXPLANATION:

Base Cases: The 0th term of the sequence is 0, and the 1st term is 1. The function directly returns these values when n is 0 or 1.

Recursive Step: For any position greater than 1, the function calculates the value by calling itself twice to add the two preceding terms together (n - 1 and n - 2).

LAB-2

a) Program to implement LINEAR SEARCH

CODE

```
1. def linear_search(arr, target):
2.     for i in range(len(arr)):
3.         if arr[i] == target:
4.             return i
5.     return -1
6.
7. my_array = [10, 23, 45, 70, 11, 15]
8. target_val = 70
9. print(linear_search(my_array,  target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: The algorithm sequentially checks each element in the array from the beginning (index 0) to the end.

Condition: During each iteration, it compares the current element to the target.

Result: If a match is found, it immediately returns the index of that element. If the loop completes and the element is never found, it returns -1 to indicate the target is not in the array.

Time Complexity: $O(n)$ because in the worst-case scenario, it has to check every single element.

b) Program to implement BINARY SEARCH

(1) With Sorted List:

CODE:

```
1. def binary_search_sorted(arr, target):
2.     low = 0
3.     high = len(arr) - 1
4.
5.     while low <= high:
6.         mid = (low + high) // 2
7.         if arr[mid] == target:
8.             return mid
9.         elif arr[mid] < target:
10.            low = mid + 1
11.        else:
12.            high = mid - 1
13.
14.    return -1
15.
16. sorted_array = [11, 22, 34, 45, 55, 67, 89]
17. target_val = 45
18. print(binary_search_sorted(sorted_array,   target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Binary search uses a divide-and-conquer approach but strictly requires the array to be sorted beforehand. It maintains a low and a high pointer to track the search space.

Condition: It calculates the mid point of the current search space.

- If the mid element is the target, it returns the mid index.
- If the target is strictly greater than the mid element, it discards the left half by moving the low pointer to mid + 1.
- If the target is strictly less than the mid element, it discards the right half by moving the high pointer to mid - 1.

Result: The loop continues narrowing the search space until low becomes greater than high. If the target isn't found by then, it returns -1.

Time Complexity: $O(\log n)$ because the search space is cut in half during every step.

(2) Without Sorted List:

CODE:

```
1. def binary_search_unsorted(arr, target):
2.     arr.sort()
3.
4.     low = 0
5.     high = len(arr) - 1
6.
7.     while low <= high:
8.         mid = (low + high) // 2
9.         if arr[mid] == target:
10.            return mid
11.        elif arr[mid] < target:
12.            low = mid + 1
13.        else:
14.            high = mid - 1
15.
16.    return -1
17.
18. unsorted_array = [67, 11, 89, 22, 55, 45, 34]
19. target_val = 45
20. print(binary_search_unsorted(unsorted_array,  target_val))
```

OUTPUT:

3

EXPLANATION:

How it works: Because binary search relies on the data being in a specific order to eliminate halves of the array, it cannot function on unsorted data.

Condition: The code first calls arr.sort() to organize the array in ascending order.

Result: Once the array is sorted, it executes the exact same binary search logic as above. *(Note: Because the array is modified and sorted before searching, the index returned corresponds to the target's position in the newly sorted array, not its original position in the unsorted input).*

Time Complexity: $O(n \log n)$ due to the initial sorting step, which dominates the $O(\log n)$ search time.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	Line 2 (Sorted): low = 6 incorrectly sets the starting index.	Change to low = 0
2.	Lines 6 & 8: mid (low + high) // 2 is missing the assignment operator.	Change to mid = (low + high) // 2
3.	Lines 12 & 14: high = mid 1 is missing a subtraction operator.	Change to high = mid - 1

WEEK-3

a) Write a Program for BUBBLE SORT

CODE:

```
1. def bubble_sort(arr):
2.     n = len(arr)
3.     for i in range(n):
4.         swapped = False
5.         for j in range(0, n - i - 1):
6.             if arr[j] > arr[j + 1]:
7.                 arr[j], arr[j + 1] = arr[j + 1], arr[j]
8.                 swapped = True
9.         if not swapped:
10.             break
11.     return arr
```

OUTPUT:

[11, 12, 22, 25, 34, 64, 90]

EXPLANATION;

Bubble sort works by repeatedly swapping adjacent elements if they are in the wrong order. With each full pass, the largest unsorted element "bubbles" up to its correct position at the end of the array.

- **The Outer Loop (for i in range(n)):** This tracks the number of passes. After the first pass, the largest element is guaranteed to be at the very end. After the second pass, the second largest is in place, and so on.
- **The Inner Loop (for j in range(0, n - i - 1)):** This loop performs the actual comparisons. It stops at n - i - 1 because we don't need to check the last i elements—they are already in their sorted, final positions from previous passes.
- **The Swap:** if arr[j] > arr[j + 1]: checks if the left element is larger than the right element. If so, it swaps them.
- **The Optimization (swapped = True/False):** If the array is already sorted, making full passes is a waste of time. The swapped variable tracks if any swaps happened during a pass. If a full pass happens with zero swaps, the loop breaks early, saving processing time.

b) Write a program for INSERTION SORT

CODE:

```
1. def insertion_sort(arr):
2.     for i in range(1, len(arr)):
3.         key = arr[i]
4.         j = i - 1
5.         while j >= 0 and key < arr[j]:
6.             arr[j + 1] = arr[j]
7.             j -= 1
8.         arr[j + 1] = key
9.     return arr
10.
11. my_array = [12, 11, 13, 5, 6]
12. print(insertion_sort(my_array))
```

OUTPUT:

[5, 6, 11, 12, 13]

EXPLANATION:

Insertion sort builds the final sorted array one item at a time. It works exactly like sorting a hand of playing cards: you take an unsorted card, figure out where it belongs in the cards you've already sorted, and insert it there.

- **The Outer Loop (for i in range(1, len(arr))):** We start at index 1 (the second element) because we assume a single element at index 0 is already "sorted". We pull this current element out and call it the key.
- **The while Loop:** We look at the elements to the left of the key (which are already sorted). As long as we haven't reached the beginning of the array (j >= 0) and the sorted element we are looking at is larger than our key (key < arr[j]), we shift that sorted element one space to the right (arr[j + 1] = arr[j]).
- **The Insertion (arr[j + 1] = key):** Once we find an element smaller than the key (or reach the start of the array), the while loop stops. We then drop the key into the empty space we created.

c) Write a program for SELECTION SORT;

CODE:

```
1. def selection_sort(arr):
2.     for i in range(len(arr)):
3.         min_idx = i
4.         for j in range(i + 1, len(arr)):
5.             if arr[j] < arr[min_idx]:
6.                 min_idx = j
7.         arr[i], arr[min_idx] = arr[min_idx], arr[i]
8.     return arr
9.
10. my_array = [64, 25, 12, 22, 11]
11. print(selection_sort(my_array))
```

OUTPUT:

[11, 12, 22, 25, 64]

EXPLANATION:

Selection sort works by dividing the array into two parts: a sorted portion at the front and an unsorted portion at the back. It repeatedly searches the unsorted portion to find the smallest element, and moves it to the end of the sorted portion.

- **The Outer Loop (for i in range(len(arr))):** This loop represents the boundary between the sorted and unsorted portions. The element at index i is where the next smallest number needs to go. We temporarily assume the element at i is the minimum (min_idx = i).
- **The Inner Loop (for j in range(i + 1, len(arr))):** This loop scans the rest of the unsorted array (everything to the right of i) to see if there is an element smaller than our current assumed minimum.
- **Updating the Minimum (if arr[j] < arr[min_idx]):** If we find a smaller element, we update min_idx to remember its location.
- **The Swap:** After checking the entire unsorted portion, we swap the element currently at position i with the actual smallest element we found at min_idx.

WEEK-4

a) Write a Program for Quick SORT

CODE:

```
1. def quicksort(arr, low, high):
2.     if low < high:
3.         i = low
4.         j = high
5.         pivot = low
6.
7.         while i < j:
8.             while (i < len(arr) and arr[i] <= arr[pivot]):
9.                 i += 1
10.
11.            while (arr[j] > arr[pivot]):
12.                j -= 1
13.
14.            if i < j:
15.                arr[i], arr[j] = arr[j], arr[i]
16.
17.        arr[j], arr[pivot] = arr[pivot], arr[j]
18.
19.        quicksort(a, low, j-1)
20.        quicksort(a, j+1, high)
21.
22. a = list(map(int, input("Enter numbers to sort: ").split(" ")))
23. n = len(a)
24. quicksort(a, 0, n-1)
25. print("arr sorted")
26. print(a)
27.
```

OUTPUT:

```
Enter numbers to sort: 6 7 8 3 4 5
arr sorted
[3, 4, 5, 6, 7, 8]
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

Algorithm Paradigm: Divide and Conquer

Pivot Selection: First element of the sub-array (arr[low])

Time Complexity:

- **Best / Average Case:** $\mathcal{O}(n \log n)$
- **Worst Case:** $\mathcal{O}(n^2)$ (when the array is already sorted or reverse sorted)

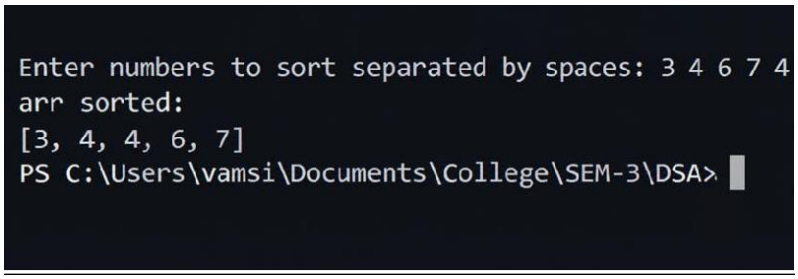
Space Complexity: $\mathcal{O}(\log n)$ auxiliary stack space for recursion.

b) Write a Program for MERG SORT:

CODE:

```
1. def merge_sort(arr):
2.     if len(arr) > 1:
3.         mid = len(arr) // 2
4.         left_half = arr[:mid]
5.         right_half = arr[mid:]
6.
7.         merge_sort(left_half)
8.         merge_sort(right_half)
9.
10.        i = j = k = 0
11.
12.        while i < len(left_half) and j < len(right_half):
13.            if left_half[i] < right_half[j]:
14.                arr[k] = left_half[i]
15.                i += 1
16.            else:
17.                arr[k] = right_half[j]
18.                j += 1
19.            k += 1
20.
21.        while i < len(left_half):
22.            arr[k] = left_half[i]
23.            i += 1
24.            k += 1
25.
26.        while j < len(right_half):
27.            arr[k] = right_half[j]
28.            j += 1
29.            k += 1
30.
31.    return arr
32.
33. user_input = input("Enter numbers to sort separated by spaces: ")
34. numbers = [int(x) for x in user_input.split()]
35. merge_sort(numbers)
36. print("arr sorted:")
37. print(numbers)
38.
```

OUTPUT:



EXPLANATION:

Algorithm Paradigm: Divide and Conquer

Merging Strategy: Out-of-place merge (requires additional memory allocation for sub-arrays)

Time Complexity:

- **Best, Average, and Worst Case:** $\mathcal{O}(n \log n)$. The array is consistently halved ($\log n$ divisions), and the merging process takes linear time (n).

Space Complexity: $\mathcal{O}(n)$ auxiliary space is required to store the temporary left_half and right_half arrays during the merging process.

Stability: Stable. Merge sort preserves the relative order of duplicate elements because it strictly checks left_half[i] < right_half[j] (elements from the left take precedence if they are equal).

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	**Unhandled ValueError** (Runtime Input)	[int(x) for x in user_input.split()] assumes perfect user behavior. If a user types letters, symbols, or floats (e.g., 5 2 a 9), the program immediately crashes. Wrap the input parsing step in a try...except ValueError: block to prompt the user again.
2.	Algorithm Logic	**No Errors Occurred.** The recursive division, list slicing, and pointer logic (i, j, k) for the merge step are perfectly executed.

WEEK-5

a) Write a Program to implement single Linked List:

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class LinkedList:
    def __init__(self):
        self.head = None

    def insert_at_start(self, data):
        new_node = Node(data)
        new_node.next = self.head
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node

    def insert_at_index(self, index, data):
        if index == 0:
            self.insert_at_start(data)
            return

        new_node = Node(data)
        current = self.head

        for _ in range(index - 1):
            if not current:
                raise IndexError("Index out of bounds")
            current = current.next

        if not current:
            raise IndexError("Index out of bounds")

        new_node.next = current.next
        current.next = new_node

    def delete_at_start(self):
        if not self.head:
            print("List is empty.")
            return
        self.head = self.head.next

    def delete_at_end(self):
        if not self.head:
            print("List is empty.")
            return
        if not self.head.next:
            self.head = None
            return

        current = self.head
        while current.next.next:
            current = current.next
        current.next = None

    def delete_at_index(self, index):
        if not self.head:
            print("List is empty.")
            return
```

```
        if index == 0:
            self.delete_at_start()
            return

        current = self.head
        for _ in range(index - 1):
            if not current.next:
                raise IndexError("Index out of bounds")
            current = current.next

        if not current.next:
            raise IndexError("Index out of bounds")

        current.next = current.next.next

    def display(self):
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" -> ".join(elements) if elements else "Empty List")

if __name__ == "__main__":
    ll = LinkedList()

    ll.insert_at_start(10)
    ll.insert_at_end(30)
    ll.insert_at_index(1, 20)
    ll.display()

    ll.delete_at_start()
    ll.display()

    ll.delete_at_end()
    ll.display()

    ll.insert_at_start(10)
    ll.insert_at_end(30)
    ll.delete_at_index(1)
    ll.display()
```

OUTPUT:

```
10 -> 20 -> 30
20 -> 30
20
10 -> 30
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

insert_at_start(data): Creates a new node, points its next reference to the current head, and updates the head to the new node.

insert_at_end(data): Traverses the list to find the last node (where next is None) and updates its reference to point to the new node. Handles empty lists by setting the head directly.

insert_at_index(index, data): Traverses to the node right before the target index (index - 1). Points the new node to the current node's next, and updates the current node to point to the new node.

delete_at_start(): Reassigns the head pointer to the second node in the list (head.next), allowing Python's garbage collector to remove the old head.

delete_at_end(): Traverses the list looking one step ahead (current.next.next). Once it finds the second-to-last node, it sets its next pointer to None.

delete_at_index(index): Traverses to the node immediately preceding the target index. It then bypasses the target node by updating the reference to current.next.next.

Data Structure Type: Linear, Dynamic (Non-contiguous memory allocation)

Time Complexity (Start Operations): $\mathcal{O}(1)$ for both insert_at_start and delete_at_start, as they only require pointer reassignment at the head.

Time Complexity (End/Index Operations): $\mathcal{O}(n)$ in the worst case, as inserting or deleting at the end or a specific index requires traversing the list from the head to the target location.

Space Complexity: $\mathcal{O}(1)$ auxiliary space per operation (no extra arrays or recursive stacks are created during insertions or deletions). Overall structure takes $\mathcal{O}(n)$ space to store the nodes.

b) Write a Program to implement Double Linked List:

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None
        self.prev = None

class DoublyLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_start(self, data):
        new_node = Node(data)
        if self.head is None:
            self.head = new_node
            return
        new_node.next = self.head
        self.head.prev = new_node
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            return
        last = self.head
        while last.next:
            last = last.next
        last.next = new_node
        new_node.prev = last

    def insert_at_index(self, index, data):
        if index == 0:
            self.insert_at_start(data)
            return

        new_node = Node(data)
        current = self.head

        for _ in range(index - 1):
            if not current:
                raise IndexError("Index out of bounds")
            current = current.next

        if not current:
            raise IndexError("Index out of bounds")

        new_node.next = current.next
        new_node.prev = current

        if current.next:
            current.next.prev = new_node
        current.next = new_node

    def delete_at_start(self):
        if not self.head:
            print("List is empty.")
            return
        if not self.head.next:
            self.head = None
            return
        self.head = self.head.next
        self.head.prev = None

    def delete_at_end(self):
        if not self.head:
            print("List is empty.")
            return
        if not self.head.next:
```

```
            self.head = None
            return

        last = self.head
        while last.next:
            last = last.next
        last.prev.next = None

    def delete_at_index(self, index):
        if not self.head:
            print("List is empty.")
            return
        if index == 0:
            self.delete_at_start()
            return

        current = self.head
        for _ in range(index):
            if not current:
                raise IndexError("Index out of bounds")
            current = current.next

        if not current:
            raise IndexError("Index out of bounds")

        if current.next:
            current.next.prev = current.prev
        if current.prev:
            current.prev.next = current.next

    def display(self):
        elements = []
        current = self.head
        while current:
            elements.append(str(current.data))
            current = current.next
        print(" <-> ".join(elements) if elements else "Empty List")


if __name__== "__main__":
    dll = DoublyLinkedList()

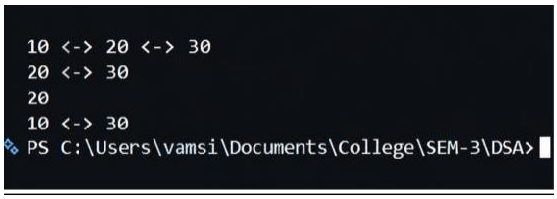
    dll.insert_at_start(10)
    dll.insert_at_end(30)
    dll.insert_at_index(1, 20)
    dll.display()

    dll.delete_at_start()
    dll.display()

    dll.delete_at_end()
    dll.display()

    dll.insert_at_start(10)
    dll.insert_at_end(30)
    dll.delete_at_index(1)
    dll.display()
```

OUTPUT:



EXPLANATION:

Data Structure Type: Linear, Dynamic, Bidirectional

Time Complexity (Start Operations): $O(1)$ for insert_at_start and delete_at_start.

Time Complexity (End/Index Operations): $O(n)$ in the worst case for traversal. *(Note: Operations at the end could be reduced to* $O(1)$ *if a tail pointer is added to the class).*

Space Complexity: $O(1)$ auxiliary space per operation.

Memory Overhead: Higher than a singly linked list. Each node requires extra memory to store the prev reference, leading to a slightly larger memory footprint.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	**Indentation SyntaxError** (Formatting Artifact)	The provided code block uses invisible non-breaking spaces instead of standard spaces for indentation. Highlighting and replacing all indents with standard keyboard spaces or tabs will allow the code to run perfectly.
2.	**Algorithm Logic**	**No Errors Occurred.** The insertion, deletion, and display methods all correctly handle doubly linked list pointers and standard edge cases (e.g., empty lists, ends of lists).

c) Write a python programme to implement Circular linked list:

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class CircularLinkedList:
    def __init__(self):
        self.head = None

    def insert_at_start(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head
        while temp.next != self.head:
            temp = temp.next

        new_node.next = self.head
        temp.next = new_node
        self.head = new_node

    def insert_at_end(self, data):
        new_node = Node(data)
        if not self.head:
            self.head = new_node
            new_node.next = self.head
            return

        temp = self.head
        while temp.next != self.head:
            temp = temp.next

        temp.next = new_node
        new_node.next = self.head

    def insert_at_index(self, index, data):
        if index == 0:
            self.insert_at_start(data)
            return

        new_node = Node(data)
        temp = self.head
        count = 0

        while temp and count < index - 1:
            temp = temp.next
            count += 1
            if temp == self.head:
                print("Index out of bounds")
                return

        new_node.next = temp.next
        temp.next = new_node

    def delete_at_start(self):
        if not self.head:
            print("List is empty")
            return

        if self.head.next == self.head:
            self.head = None
            return

        temp = self.head
        while temp.next != self.head:
```

```
            temp = temp.next

        temp.next = self.head.next
        self.head = self.head.next

    def delete_at_end(self):
        if not self.head:
            print("List is empty")
            return

        if self.head.next == self.head:
            self.head = None
            return

        temp = self.head
        while temp.next.next != self.head:
            temp = temp.next

        temp.next = self.head

    def delete_at_index(self, index):
        if not self.head:
            print("List is empty")
            return

        if index == 0:
            self.delete_at_start()
            return

        temp = self.head
        count = 0

        while temp and count < index - 1:
            temp = temp.next
            count += 1
            if temp.next == self.head:
                print("Index out of bounds")
                return

        temp.next = temp.next.next

    def display(self):
        if not self.head:
            print("List is empty")
            return
        temp = self.head
        while True:
            print(temp.data, end=" -> ")
            temp = temp.next
            if temp == self.head:
                break
        print(f"(back to head: {self.head.data})")

if __name__== "__main__":
    cll = CircularLinkedList()

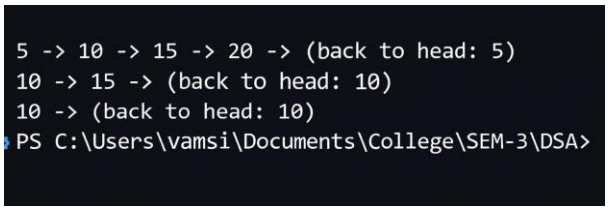
    cll.insert_at_end(10)
    cll.insert_at_end(20)
    cll.insert_at_start(5)
    cll.insert_at_index(2, 15)
    cll.display()

    cll.delete_at_start()
    cll.delete_at_end()
    cll.display()

    cll.delete_at_index(1)
    cll.display()

```

OUTPUT:



EXPLANATION:

Time Complexity (Start Operations): $\mathcal{O}(n)$ for both insert_at_start and delete_at_start in this specific implementation. Because the list is circular and we only track the head pointer, we have to traverse the entire list to find the very last node to update its next pointer. *(Note: You can easily drop this to* $\mathcal{O}(1)$ *if you modify the code to maintain a tail pointer alongside the head!)*

Time Complexity (End/Index Operations): $\mathcal{O}(n)$ in the worst case, as inserting or deleting at the end or a specific index requires traversing the list from the head to the target location.

Space Complexity: $\mathcal{O}(1)$ auxiliary space per operation (no extra arrays or recursive stacks are created during insertions or deletions). The overall structure takes $\mathcal{O}(n)$ space to store the actual nodes.

WEEK-6

a) Write a Program to implement Stack using Array:

CODE

```
class Stack:
    def __init__(self):
        self.stack = []

    def push(self, item):
        self.stack.append(item)
        print(f"Pushed {item} to stack")

    def pop(self):
        if self.is_empty():
            print("Stack Underflow")
        else:
            print(f"Popped {self.stack.pop()} from stack")

    def peek(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            print(f"Top element is {self.stack[-1]}")

    def is_empty(self):
        return len(self.stack) == 0

    def display(self):
        if self.is_empty():
            print("Stack is empty")
        else:
            print("Stack elements:", self.stack[::-1])

    def menu(self):
        while True:
            print("\n1. Push")
            print("2. Pop")
            print("3. Peek")
            print("4. Display")
            print("5. Exit")

            choice = input("Enter your choice (1-5): ")

            if choice == '1':
                item = input("Enter item to push: ")
                self.push(item)
            elif choice == '2':
                self.pop()
            elif choice == '3':
                self.peek()
            elif choice == '4':
                self.display()
            elif choice == '5':
                print("Exiting...")
                break
            else:
                print("Invalid choice. Please try again.")

my_stack = Stack()
my_stack.menu()

```

OUTPUT:

```
1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter item to push: 5
Pushed 5 to stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 1
Enter item to push: 3
Pushed 3 to stack

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 4
Stack elements: ['3', '5', '4']

1. Push
2. Pop
3. Peek
4. Display
5. Exit
Enter your choice (1-5): 5
Exiting...
```

EXPLANATION:

The provided code implements a Stack data structure using a standard Python list within a single class. It operates on the Last-In, First-Out (LIFO) principle, meaning the most recently added item is the first one removed. The class bundles essential operations like push (adding items), pop (removing items), and peek (viewing the top item), along with an interactive menu method. By creating a Stack object and calling this menu, users can manage the stack directly from the command line in a continuous loop.

b) Write python program to implement stack using LinkedList

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.next = None

class Stack:
    def _init_(self):
        self.top = None

    def push(self, data):
        new_node = Node(data)
        new_node.next = self.top
        self.top = new_node
        print(f"{data} is added to stack")

    def pop(self):
        if self.top is None:
            print("Stack underflow")
        else:
            print(f"{self.top.data} is deleted from stack")
            self.top = self.top.next

    def peek(self):
        if self.top is None:
            print("No elements in the stack")
        else:
            print(self.top.data)

    def display(self):
        if self.top is None:
            print("Empty Stack")
        else:
            current = self.top
            while current:
                print(current.data, end="->")
                current = current.next
            print(None)

s = Stack()
while (True):
    print("1.Push")
    print("2.Pop")
    print("3.Peek")
    print("4.Display")
    print("5.Exit")
    i = int(input("Enter your choice: "))
    if (i == 1):
        e = int(input("Enter element to add: "))
        s.push(e)
    elif (i == 2):
        s.pop()
    elif (i == 3):
        s.peek()
    elif (i == 4):
        s.display()
    elif (i == 5):
        exit()
    else:
        print("Choose correct choice")

```

OUTPUT:

```
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 6
6 is added to stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 8
8 is added to stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 4
8->6->5->2->None
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 2
8 is deleted from stack
1.Push
2.Pop
3.Peek
4.Display
5.Exit
Enter your choice: 4
6->5->2->None
```

EXPLANATION:

push(data): Creates a new node, points its next reference to the current top node, and updates the top reference to point to this new node.

pop(): Checks for an empty stack (underflow). Reassigns the top pointer to the next node in the stack (top.next), allowing Python's garbage collector to remove the old top node.

peek(): Evaluates if the stack is empty. If not, it retrieves and outputs the data stored in the current top node without altering any node references.

display(): Traverses the stack starting from the top node, following the next pointers until it reaches the end (None), printing the data of each node along the way to represent the stack from top to bottom.

Data Structure Type: Linear, Dynamic (Non-contiguous memory allocation), operating on a Last-In, First-Out (LIFO) principle.

Time Complexity (Push, Pop, Peek Operations): O(1) for push, pop, and peek, as they only require direct pointer reassignment or data access at the head (top) of the linked list without any traversal.

Time Complexity (Display Operation): O(n) in the worst case, as displaying the full stack requires traversing every node from the top down to the bottom.

Space Complexity: O(1) auxiliary space per operation (no extra arrays or recursive stacks are created during pushes or pops). The overall structure takes O(n) space to dynamically store the nodes.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	**AttributeError: 'NoneType' object has no attribute 'data'**	Occurs if attempting to access self.top.data or self.top.next when the stack is empty. Ensure if self.top is None: (or is_empty()) is checked before trying to retrieve or modify the top node's attributes.
2.	**Infinite Loop in display()**	Caused by forgetting to advance the traversal pointer (e.g., omitting current = current.next) inside the while loop. Add the pointer update step to ensure the loop progresses and eventually terminates when current becomes None.

WEEK-7

a) Write python program to implement Queue using

Arrays CODE:

```
1. class Queue:
2.     def __init__(self,size):
3.         self.size = size
4.         self.queue = [None]*size
5.         self.front = -1
6.         self.rear = -1
7.     def enqueue(self,data):
8.         if  (self.rear>=self.size-1):
9.             print("Queue overflow")
10.        else:
11.            if (self.front == -1):
12.                self.front = 0
13.            self.rear+=1
14.            self.queue[self.rear]=data
15.            print(f"{data} is added to queue")
16.    def dequeue(self):
17.        if (self.front>self.rear or self.front==-1):
18.            print("Queue underflow")
19.        else :
20.            print(f"{self.queue[self.front]} is deleted from queue")
21.            self.queue[self.front] = None
22.            self.front+=1
23.    def peek(self):
24.        if (self.front==-1):
25.            print("No elements in the queue")
```

```
26.       else:
27.           print(self.queue[self.front])
28.    def display(self):
29.       if (self.front==-1):
30.           print("Empty Queue")
31.       else:
32.           for i in range (0,self.rear+1):
33.               print(self.queue[i],end="->")
34.           print(None)
35. q=Queue(5)
36. while (True):
37.    print("1.Enqueue")
38.    print("2.Dequeue")
39.    print("3.peek")
40.    print("4.Display")
41.    print("5.Exit")
42.    i = int(input("Enter your chocice: "))
43.    if (i==1):
44.       e = int(input("Enter element to add"))
45.       q.enqueue(e)
46.    elif(i==2):
47.       q.dequeue()
48.    elif(i==3):
49.       q.peek()
50.    elif(i==4):
51.       q.display()
52.    elif(i==5):
53.       exit()
```

```
54.    else:
55.        print("Choose correct
choice") 56.
```

OUTPUT:

```
Enter your chocice: 1
Enter element to add5
5 is added to queue
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your chocice: 1
Enter element to add7
7 is added to queue
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your chocice: 3
3
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your chocice: 2
3 is deleted from queue
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your chocice: 4
None->5->7->None
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your chocice: 5
PS C:\Users\varun\Documents\College\SEM-3\DSA>
```

EXPLANATION:

enqueue(data): Checks for queue overflow (rear >= size - 1). If the queue is empty (front == -1), it initializes front to 0. It then increments the rear index by 1 and inserts the data at the rear position in the fixed-size array.

dequeue(): Checks for queue underflow (front > rear or front == -1). If not empty, it prints and clears the element at the front index by setting it to None, then increments the front index to point to the next element.

peek(): Evaluates if the queue is empty (front == -1). If not, it retrieves and outputs the element located at the front index without modifying any pointer or array values.

display(): Checks if the queue is empty (front == -1). If not empty, it iterates through the internal array from index 0 up to rear, printing each element in sequence to display the current state of the queue array.

Data Structure Type: Linear, Static/Fixed-Capacity Array-Based, operating on a First-In, First-Out (FIFO) principle.

Time Complexity (Enqueue, Dequeue, Peek Operations): $O(1)$ for enqueue, dequeue, and peek, as they perform direct array index access and basic variable updates without iteration.

Time Complexity (Display Operation): $O(n)$ in the worst case (where n is rear + 1), as displaying the queue contents requires traversing elements from index 0 up to the rear pointer.

Space Complexity: $O(1)$ auxiliary space for individual operations. Overall space complexity is $O(N)$ based on the fixed maximum capacity (size) allocated during initialization.

b) Write python program to implement Queue using Linked List CODE:

```
1. class Node:
2.     def __init__(self, data):
3.         self.data = data
4.         self.next = None
5. class Queue:
6.     def __init__(self):
7.         self.front = None
8.         self.rear = None
9.     def enqueue(self, data):
10.        curr = Node(data)
11.        if self.rear is None:
12.            self.front = self.rear = curr
13.            return
14.        self.rear.next = curr
15.        self.rear = curr
16.    def dequeue(self):
17.        if self.front is None:
18.            print("Queue underflow")
19.            return
20.
21.        self.front = self.front.next
22.        if self.front is None:
23.            self.rear = None
24.    def peek(self):
25.        if self.front is None:
26.            print("No elements in queue")
```

```
27.       else:
28.           print(self.front.data)
29.   def display(self):
30.       if self.front is None:
31.           print("No elements in queue")
32.       else:
33.           temp = self.front
34.           while temp is not None:
35.               print(temp.data, end="->")
36.               temp = temp.next
37.           print("None")
38.
39. q = Queue()
40. while (True):
41.   print("1.Enqueue")
42.   print("2.Dequeue")
43.   print("3.peek")
44.   print("4.Display")
45.   print("5.Exit")
46.   i = int(input("Enter your chocice: "))
47.   if (i==1):
48.       e = int(input("Enter element to add"))
49.       q.enqueue(e)
50.   elif(i==2):
51.       q.dequeue()
52.   elif(i==3):
53.       q.peek()
54.   elif(i==4):
```

```
55.      q.display()
56.    elif(i==5):
57.      exit()
58.    else:
59.      print("Choose correct
choice") 60.
```

OUTPUT:

```
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add4
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add5
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add3
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your choice: 3
4
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your choice: 4
4->8->3->None
1.Enqueue
2.Dequeue
3.peek
4.Display
5.Exit
Enter your choice: 5
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

enqueue(data): Creates a new node containing the data. If the queue is empty (rear is None), both front and rear are set to point to this new node. Otherwise, it updates the next pointer of the current rear node to the new node and shifts rear to point to the new node.

dequeue(): Checks for queue underflow (front is None). If not empty, it advances the front pointer to front.next, removing the references to the old head node so Python's garbage collector can reclaim it. If front becomes None after removal, rear is also set to None to reset the empty queue.

peek(): Evaluates if the queue is empty (front is None). If not, it retrieves and outputs the data stored in the current front node without altering any node references.

display(): Traverses the queue starting from the front node, following the next pointers until reaching None, printing the data of each node along the way to represent the
queue from front to rear.

Data Structure Type: Linear, Dynamic (Non-contiguous memory allocation), operating on a First-In, First-Out (FIFO) principle.

Time Complexity (Enqueue, Dequeue, Peek Operations): $O(1)$ for enqueue, dequeue, and peek, as they only require direct pointer reassignments or data access at the front or rear nodes without any traversal.

Time Complexity (Display Operation): $O(n)$ in the worst case, as displaying the full queue requires traversing every node from the front down to the rear.

Space Complexity: $O(1)$ auxiliary space per operation (no extra arrays or recursive stacks created during enqueues or dequeues). The overall structure takes $O(n)$ space to dynamically store the nodes.

ERROR TABLE:

SNO	ERROR	SOLUTION
1.	AttributeError: 'NoneType' object has no attribute 'data' (or 'next')	Occurs if attempting to access self.front.data or self.front.next when the queue is empty (self.front is None). Ensure if self.front is None: is checked before trying to retrieve or modify the front node's attributes.
2.	Dangling rear Pointer / Memory Leak	Occurs during dequeue() if self.rear is not reset to None when the last element is removed (self.front becomes None). Ensure if self.front is None: self.rear = None is included so rear does not point to a stale node.
3.	Unlinked Nodes / Broken Chain in enqueue()	Caused by forgetting self.rear.next = curr before reassigning self.rear = curr

		when adding to a non-empty queue. Always update the current rear node's next reference to point to the new node before updating self.rear

c) Write python program to implement Circular Queue using

Arrays CODE:

```
1. class CircularQueue:
2.     def __init__(self, size):
3.         self.size = size
4.         self.queue = [None] * size
5.         self.front = -1
6.         self.rear = -1
7.
8.     def enqueue(self, data):
9.         if (self.rear + 1) % self.size == self.front:
10.            print("Queue overflow")
11.            return
12.
13.        if self.front == -1:
14.            self.front = 0
15.
16.        self.rear = (self.rear + 1) % self.size
17.        self.queue[self.rear] = data
18.        print(f"{data} is added to
queue") 19.
20.    def dequeue(self):
21.        if self.front == -1:
22.            print("Queue underflow")
23.            return
```

```
24.
25.       print(f"{self.queue[self.front]} is deleted from queue")
26.       self.queue[self.front] = None
27.
28.       if self.front == self.rear:
29.          self.front = -1
30.          self.rear = -1
31.       else:
32.          self.front = (self.front + 1) %
self.size 33.
34.    def peek(self):
35.       if self.front == -1:
36.          print("No elements in the queue")
37.       else:
38.          print(self.queue[self.front])
39.
40.    def display(self):
41.       if self.front == -1:
42.          print("Empty Queue")
43.          return
44.       temp = self.front
45.       while True:
46.          print(self.queue[temp], end=" -> ")
47.          if temp == self.rear:
48.             break
49.          temp = (temp + 1) % self.size
50.       print("None")
51.
```

```
52. q = CircularQueue(5)
53. while True:
54.     print("\n1.Enqueue\n2.Dequeue\n3.Peek\n4.Display\n5.Exit")
55.     try:
56.         choice = int(input("Enter your choice: "))
57.         if choice == 1:
58.             e = int(input("Enter element to add: "))
59.             q.enqueue(e)
60.         elif choice == 2:
61.             q.dequeue()
62.         elif choice == 3:
63.             q.peek()
64.         elif choice == 4:
65.             q.display()
66.         elif choice == 5:
67.             break
68.         else:
69.             print("Choose a valid choice")
70.     except ValueError:
71.         print("Please enter a valid
number") 72.
```

OUTPUT:

```
1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 4
4 is added to queue

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 1
Enter element to add: 8
8 is added to queue

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 4
2 -> 4 -> 8 -> None

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 3
2

1.Enqueue
2.Dequeue
3.Peek
4.Display
5.Exit
Enter your choice: 5
PS C:\Users\vamsi\Documents\College\SEM-3\DSA>
```

EXPLANATION:

enqueue(data): Checks for queue overflow using circular indexing (rear + 1) % size == front. If the queue is empty (front == -1), it initializes front to 0. It then advances rear circularly using (rear + 1) % size and inserts the data at the new rear position in the fixed-size array.

dequeue(): Checks for queue underflow (front == -1). If not empty, it prints and clears the element at the front index by setting it to None. If front == rear (removing the last remaining element), it resets both front and rear back to -1; otherwise, it advances front circularly using (front + 1) % size.

peek(): Evaluates if the queue is empty (front == -1). If not, it retrieves and outputs the element located at the front index without modifying any pointers or array values.

display(): Checks if the queue is empty (front == -1). If not, starting from front, it traverses each index circularly using (temp + 1) % size until it reaches rear, printing each element in sequence to represent the circular queue from front to rear.

Data Structure Type: Linear, Static/Fixed-Capacity Array-Based (Circular Memory Reuse), operating on a First-In, First-Out (FIFO) principle.

Time Complexity (Enqueue, Dequeue, Peek Operations): $O(1)$ for enqueue, dequeue, and peek, as they perform direct array index access and modulo pointer arithmetic without any traversal.

Time Complexity (Display Operation): $O(n)$ in the worst case (where n is the number of active elements in the queue), as displaying the queue contents requires traversing every element from front to rear.

Space Complexity: $O(1)$ auxiliary space for individual operations. Overall space complexity is $O(N)$ based on the fixed maximum capacity (size) allocated during initialization.

LAB-8

a) write a python program to implement a binary tree with its operations using a array

CODE:

```
class BinaryTreeArray:
    def __init__(self, size):
        self.tree = [None] * size
        self.size = size

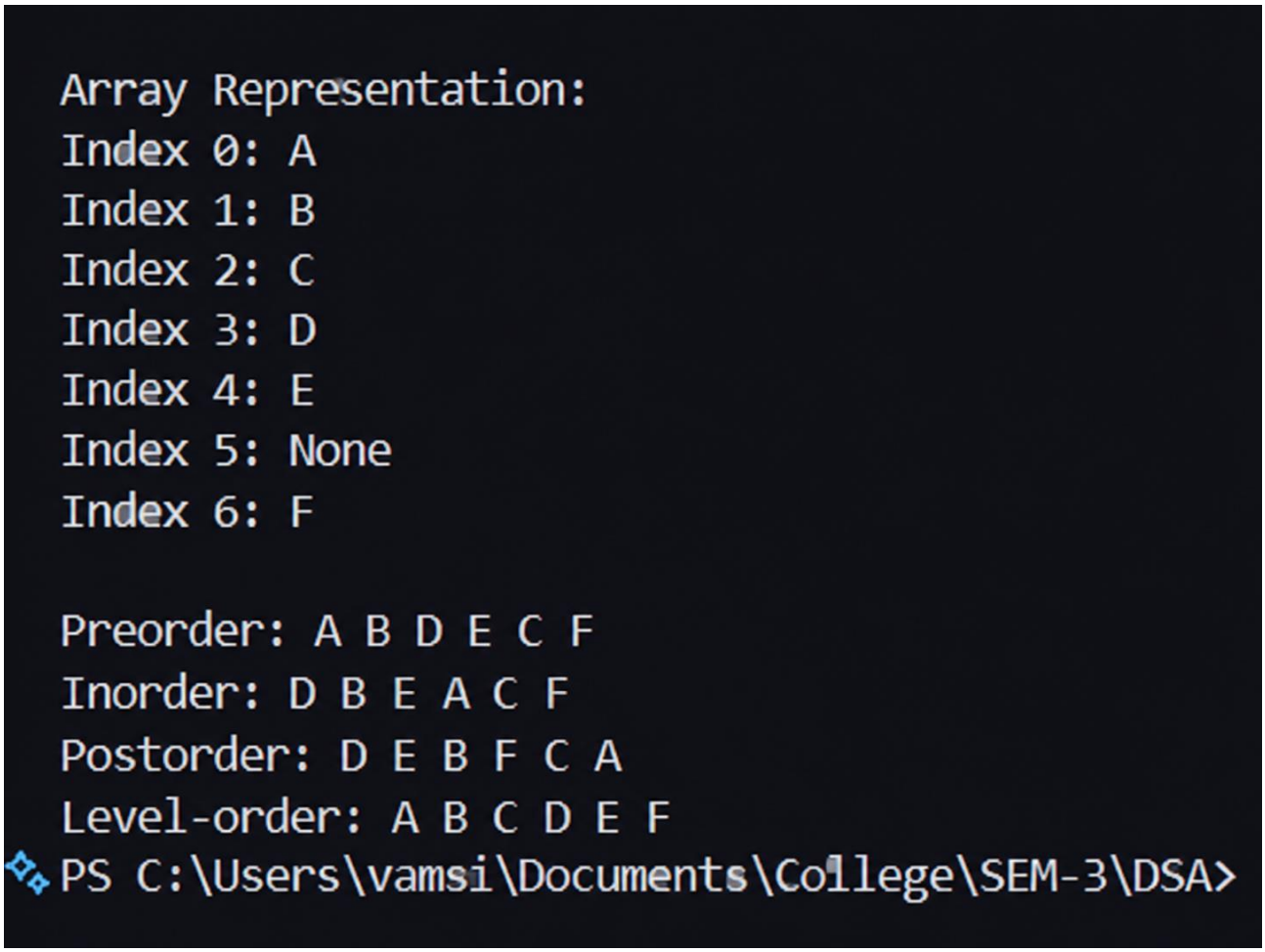
# Insert root
    def set_root(self, data):
        self.tree[0] = data
# Set left child
    def set_left(self, parent_index, data):
        child_index = 2 * parent_index + 1
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")
# Set right child
    def set_right(self, parent_index, data):
        child_index = 2 * parent_index + 2
        if child_index < self.size:
            self.tree[child_index] = data
        else:
            print("Index out of range!")
    # Preorder Traversal
    def preorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        print(self.tree[index], end=" ")
        self.preorder(2 * index + 1)
        self.preorder(2 * index + 2)

# Inorder Traversal
    def inorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.inorder(2 * index + 1)
        print(self.tree[index], end=" ")
        self.inorder(2 * index + 2)
# Postorder Traversal
    def postorder(self, index=0):
        if index >= self.size or self.tree[index] is None:
            return
        self.postorder(2 * index + 1)
        self.postorder(2 * index + 2)
```

```
        print(self.tree[index], end=" ")
# Level-order Traversal
    def level_order(self):
        for value in self.tree:
            if value is not None:
                print(value, end=" ")
# Display Array
    def display(self):
        print("\nArray Representation:")
        for i in range(self.size):
            print(f"Index {i}: {self.tree[i]}")
# MAIN PROGRAM
tree = BinaryTreeArray(7)
# Create the tree
tree.set_root('A')
# Children of A
tree.set_left(0, 'B')
tree.set_right(0, 'C')
# Children of B
tree.set_left(1, 'D')
tree.set_right(1, 'E')
# Right child of C
tree.set_right(2, 'F')
# Display tree
tree.display()
# Traversals
print("\nPreorder: ", end="")
tree.preorder()
print("\nInorder: ", end="")
tree.inorder()
print("\nPostorder: ", end="")
tree.postorder()
print("\nLevel-order: ", end="")
tree.level_order()

```

OUTPUT:



EXPLANATION:

push(data):

Creates a new node containing the data. If the stack is empty (top is None), it initializes top to point to the new node. Otherwise, it links the new node to the rest of the stack by setting new_node.next = self.top, then updates top to point to the newly added node, making it the newest element at the top of the stack.

pop():

Checks for stack underflow (top is None). If not empty, it accesses the top node and captures its data. It then removes the node by updating self.top = self.top.next, bypassing the current top node and making the next element the new top, allowing

Python's garbage collector to safely remove the old node.

peek():

Evaluates if the stack is empty (top is None). If not, it accesses the top node and outputs its data without altering any node references or removing the element from the data structure.

display():

Checks if the stack is empty (top is None). If not, starting from the top node, it iterates through each node using temp.next until temp reaches None, printing the data of each node in sequential order from top to bottom to represent the current state of the stack.

Data Structure Type: Linear, Dynamic (Non-contiguous memory allocation with singular linkage), operating on a Last-In, First-Out (LIFO) principle.

Time Complexity (Push, Pop, Peek Operations): O(1) for push, pop, and peek, as maintaining a single top pointer allows instant insertion, deletion, and access directly at the head of the list without requiring any linear traversal.

Time Complexity (Display Operation): O(n) in the worst case, as displaying the full stack requires traversing every node sequentially from the top pointer down to the base of the stack.

Space Complexity: O(1) auxiliary space per operation. The overall structure takes O(n) space to dynamically store the nodes.

b) write a python program to implement a binary tree with its operations using a linked list

CODE:

```
class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

# BINARY TREE CLASS
class BinaryTree:
    def __init__(self):
        self.root = None

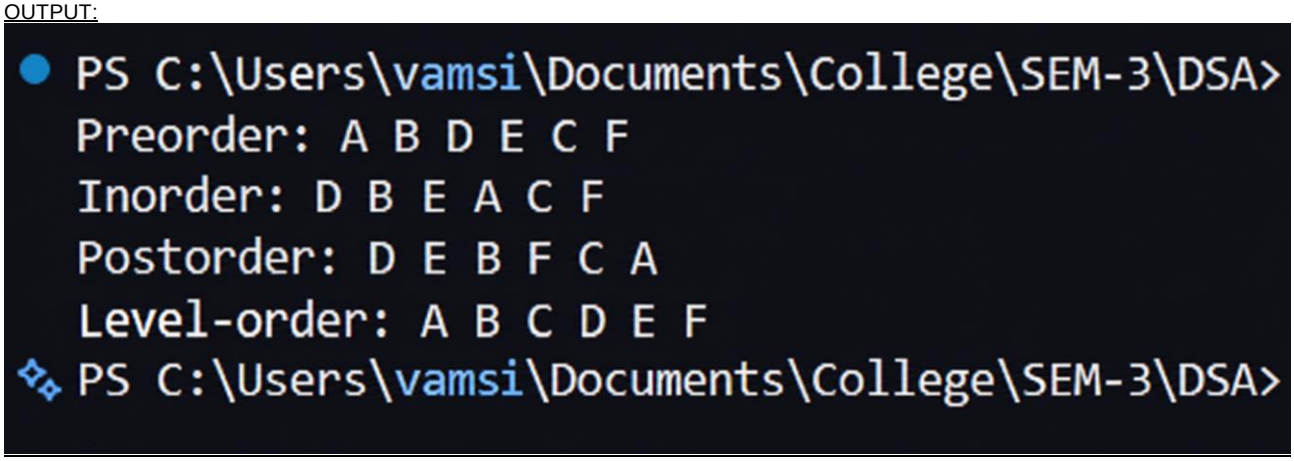
# Create the binary tree
    def create_tree(self):
        self.root = Node('A')
        # Left and right children of A
        self.root.left = Node('B')
        self.root.right = Node('C')
        # Children of B
        self.root.left.left = Node('D')
        self.root.left.right = Node('E')
        # Right child of C
        self.root.right.right = Node('F')
    # Preorder Traversal
    def preorder(self, node):
        if node is None:
            return
        print(node.data, end=" ")
        self.preorder(node.left)
        self.preorder(node.right)

# Inorder Traversal
    def inorder(self, node):
        if node is None:
            return
        self.inorder(node.left)
        print(node.data, end=" ")
        self.inorder(node.right)
# Postorder Traversal
    def postorder(self, node):
        if node is None:
            return
        self.postorder(node.left)
        self.postorder(node.right)
        print(node.data, end=" ")
# Level-order Traversal
    def level_order(self):
        if self.root is None:
            return
        queue = [self.root]
        while queue:
            node = queue.pop(0)
            print(node.data, end=" ")
            # Add left child
            if node.left is not None:
                queue.append(node.left)
            # Add right child
            if node.right is not None:
                queue.append(node.right)

# MAIN PROGRAM
tree = BinaryTree()
# Create the binary tree
tree.create_tree()
# Traversals
print("Preorder: ", end="")
tree.preorder(tree.root)
print("\nInorder: ", end="")
tree.inorder(tree.root)
```

```
69. print("\nPostorder: ", end="")
70. tree.postorder(tree.root)
71. print("\nLevel-order: ", end="")
72. tree.level_order()
73.
```

OUTPUT:



EXPLANATION:

Here is the explanation for the **Linked Node-based Binary Tree** code, formatted similarly so you can easily paste it into your Word file:

Node and BinaryTree Initialization: Defines the fundamental building blocks of the structure. The Node class creates independent objects containing the data alongside two pointers (left and right) initialized to None. The BinaryTree class manages the overall structure by maintaining a root pointer, initially set to None, which serves as the primary entry point to access the hierarchical data.

create_tree(): Manually constructs a specific binary tree structure by instantiating Node objects and establishing parent-child links. It sets 'A' as the root, links 'B' and 'C' as its left and right children respectively, and continues linking subsequent child nodes ('D', 'E', 'F'), dynamically allocating memory for each discrete node.

preorder(node): Executes a depth-first traversal using recursion. If the node is not None, it processes (prints) the current node's data immediately, then recursively traverses the entire left subtree, followed by the entire right subtree. This follows a **Root-Left-Right** sequence.

inorder(node): Executes a depth-first traversal using recursion. It recursively dives down the left subtree until it reaches a null node, processes (prints) the current node's data, and finally traverses the right subtree. This follows a **Left-Root-Right** sequence.

postorder(node): Executes a depth-first traversal using recursion. It fully explores the left subtree, then fully explores the right subtree, and only processes (prints) the current node's data after its children have been visited. This follows a **Left-Right-Root** sequence.

level_order(): Executes a breadth-first traversal using an iterative queue approach. It verifies the tree is not empty, then initializes a list (acting as a queue) starting with the root node. In a loop, it removes (pop(0)) the front node, prints its data, and appends its existing left and right children to the back of the queue. This ensures nodes are processed horizontally, level by level, from top to bottom and left to right.

Data Structure Type: Hierarchical, Dynamic (Non-contiguous memory allocation using singular references/pointers for child nodes), operating as a non-linear binary tree structure.

Time Complexity (All Traversals): O(n), where *n* is the total number of nodes. Every traversal method (Preorder, Inorder, Postorder, and Level-order) visits and processes each node in the tree exactly once.

Space Complexity:

- **O(h)** for the depth-first traversals (Preorder, Inorder, Postorder), where *h* is the height of the tree. This space is used by the system's call stack to manage the recursive function calls.
- **O(w)** for the Level-order traversal, where *w* is the maximum width of the tree (the most nodes present on any single level). This space is required to dynamically store the nodes in the auxiliary queue list.

LAB-9

write a python program to implement a binary Search Tree

CODE:

```
# Binary Search Tree

class Node:
    def __init__(self, data):
        self.data = data
        self.left = None
        self.right = None

class BinarySearchTree:

    # Insert a node
    def insert(self, root, data):
        if root is None:
            return Node(data)

        if data < root.data:
            root.left = self.insert(root.left, data)
        elif data > root.data:
            root.right = self.insert(root.right, data)

        return root

    # Search for a node
    def search(self, root, data):
        if root is None:
            return False

        if root.data == data:
            return True
        elif data < root.data:
            return self.search(root.left, data)
        else:
            return self.search(root.right, data)

    # Find minimum value node
    def find_min(self, root):
        current = root

        while current.left is not None:
            current = current.left

        return current

```

```
    # Delete a node
    def delete(self, root, data):
        if root is None:
            return root

        if data < root.data:
            root.left = self.delete(root.left, data)

        elif data > root.data:
            root.right = self.delete(root.right, data)

        else:
            # Case 1: No child
            if root.left is None and root.right is None:
                return None

            # Case 2: Only right child
            elif root.left is None:
                return root.right

            # Case 3: Only left child
            elif root.right is None:
                return root.left

            # Case 4: Two children
            else:
                temp = self.find_min(root.right)
                root.data = temp.data
                root.right = self.delete(root.right, temp.data)

        return root

    # Inorder: Left -> Root -> Right
    def inorder(self, root):
        if root is not None:
            self.inorder(root.left)
            print(root.data, end=" ")
            self.inorder(root.right)

    # Preorder: Root -> Left -> Right
    def preorder(self, root):
        if root is not None:
            print(root.data, end=" ")
            self.preorder(root.left)
            self.preorder(root.right)

```

```
    # Postorder: Left -> Right -> Root
    def postorder(self, root):
        if root is not None:
            self.postorder(root.left)
            self.postorder(root.right)
            print(root.data, end=" ")

# Main program
bst = BinarySearchTree()
root = None

while True:
    print("\n\n--- BINARY SEARCH TREE ---")
    print("1. Insert")
    print("2. Search")
    print("3. Delete")
    print("4. Inorder Traversal")
    print("5. Preorder Traversal")
    print("6. Postorder Traversal")
    print("7. Exit")

    choice = int(input("Enter your choice: "))

    if choice == 1:
        data = int(input("Enter value to insert: "))
        root = bst.insert(root, data)
        print("Node inserted successfully.")

    elif choice == 2:
        data = int(input("Enter value to search: "))

        if bst.search(root, data):
            print("Element found.")
        else:
            print("Element not found.")

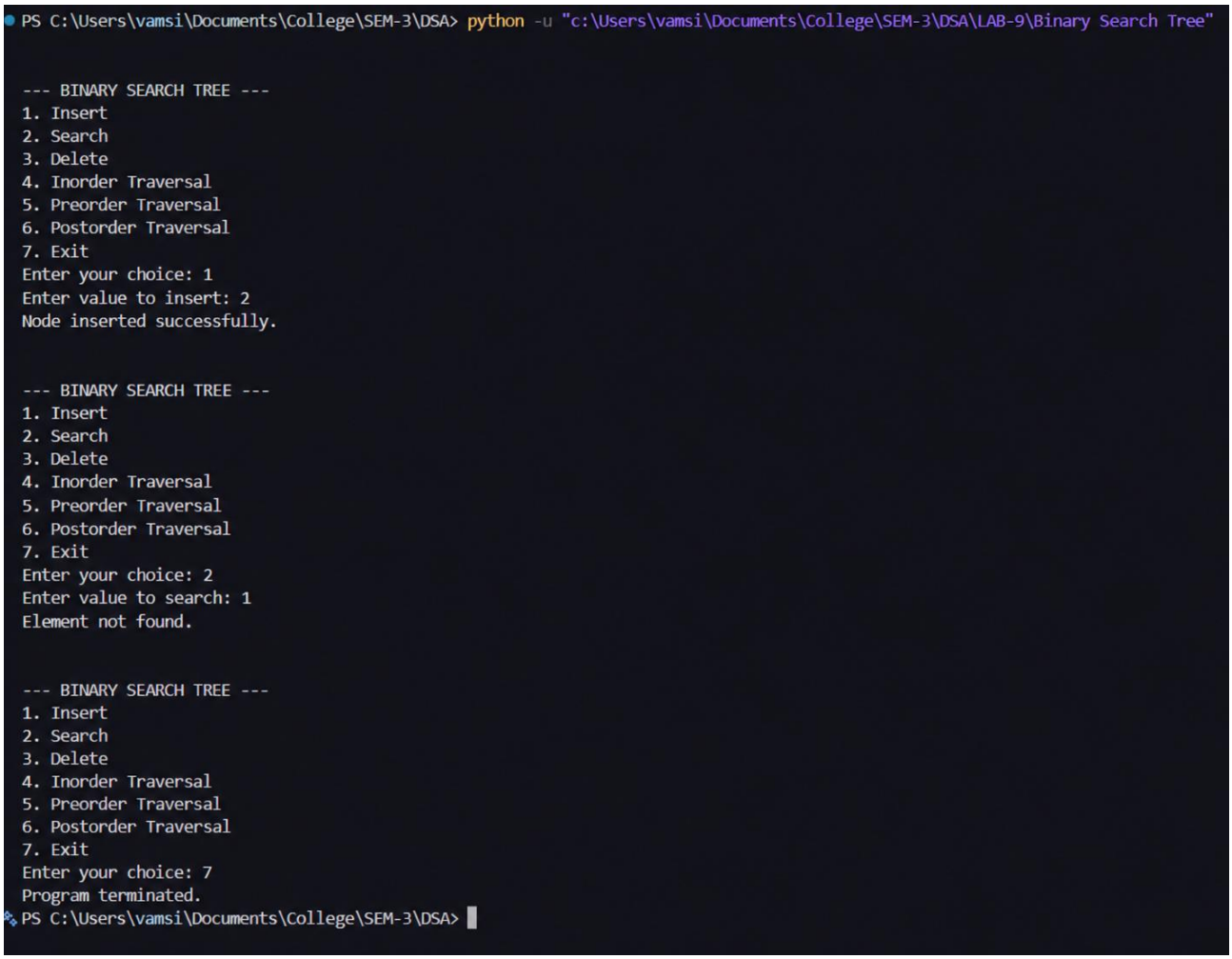
    elif choice == 3:
        data = int(input("Enter value to delete: "))
        root = bst.delete(root, data)
        print("Node deleted successfully.")

    elif choice == 4:
        print("Inorder Traversal:")
        bst.inorder(root)

    elif choice == 5:
```

```
136.         print("Preorder Traversal:")
137.         bst.preorder(root)
138.
139.     elif choice == 6:
140.         print("Postorder Traversal:")
141.         bst.postorder(root)
142.
143.     elif choice == 7:
144.         print("Program terminated.")
145.         break
146.
147.     else:
148.         print("Invalid choice.")
149.
```

OUTPUT:



EXPLANATION:

1. **Node Class**: Acts as the blueprint for individual tree elements. Each object initializes with a data value and sets its left and right child pointers to None.
2. **BinarySearchTree Class**: Encapsulates all tree manipulations using standard recursive algorithms:

- **Insertion (insert)**: Recursively navigates down the tree depending on whether the input value is smaller or larger than the current node's value, attaching a new Node once an empty spot (None) is reached.
- **Search (search)**: Traverses down specific branches based on value comparisons, returning True if a matching node is found and False if it hits a None leaf without finding the target.
- **Deletion (delete)**: Handles node removal across four distinct structural conditions: leaf nodes (no children), single-child nodes (replacing the node with its active child), and two-children nodes (replacing the target node's value with the minimum value from its right subtree, retrieved via find_min, then deleting that duplicate minimum node).
- **Tree Traversals (inorder, preorder, postorder)**: Demonstrates the three depth-first search strategies:
  - *Inorder* (Left →Root →Right) outputs elements in strictly sorted ascending order.
  - *Preorder* (Root →Left →Right) visits root nodes first, useful for cloning or serializing trees.
  - *Postorder* (Left →Right →Root) visits child nodes before their parents, commonly used for bottom-up cleanup operations.

3. **Interactive Main Program**: Uses a standard console loop (while True) with user inputs to dynamically call the BST methods and retain state across operations via the root reference (root).